

Intégration de tests automatisés Playwright dans un pipeline Bamboo

Guide pratique

Objectif : montrer comment utiliser Playwright comme contrôle automatique après un déploiement QA, tout en gardant une stratégie de tests simple, progressive et maintenable.

Compétences illustrées : Playwright • TypeScript • Bamboo • Git • CI/CD • Stratégie de tests

1. Objectif du pipeline

Le principe est de valider automatiquement qu'une application déployée en environnement QA reste utilisable. Après le déploiement, Bamboo exécute un test de fumée Playwright. Si ce test échoue, le déploiement est considéré comme en échec.

Flux général

Git → Build Bamboo → Déploiement QA → Test Playwright → Verdict

Cette approche place le test automatisé au plus près du déploiement. Elle permet de détecter rapidement une régression critique, par exemple l'impossibilité de se connecter à l'application.

Principe d'architecture

- L'application est déployée sur l'environnement QA.
- Node.js et Playwright s'exécutent sur l'agent Bamboo, pas sur la VM applicative.
- L'agent doit pouvoir joindre l'application QA en HTTPS.
- Un premier test de fumée suffit pour valider le mécanisme avant d'élargir la couverture.

2. Organisation recommandée des tests

Une structure claire sépare les niveaux de tests et le code réutilisable. Les tests unitaires restent avec le code applicatif ; Playwright est utilisé pour les tests qui pilotent l'application déployée.

```
Tests/PlaywrightTests/
├── src/
│   ├── pages/           # Page Objects
│   ├── fixtures/        # Fixtures partagées
│   ├── api/             # Clients API
│   └── utils/           # Helpers
├── test-data/          # Jeux de données
└── tests/
    ├── 01-smoke/
    ├── 02-integration/
    ├── 03-system/
    ├── 04-acceptance/
    └── 05-non-functional/
```

Pourquoi cette structure ?

Catégorie	But	Moment d'exécution
Smoke	Vérifier les fonctions vitales	Après chaque déploiement QA
Intégration	Vérifier les échanges entre composants	Selon disponibilité des services
Système	Tester les fonctionnalités complètes	Nuit / avant release
Acceptation	Valider un parcours métier	Avant release
Non fonctionnel	Accessibilité, performance, sécurité	Selon le besoin

3. Tags et stratégie d'exécution

La régression et le re-test ne nécessitent pas de dossiers séparés. Ils peuvent être sélectionnés avec des tags tout en conservant chaque scénario dans son niveau de test.

Tag	Usage	Commande
@smoke	Fonctions vitales	<code>npx playwright test --grep @smoke</code>
@regression	Suite de non-régression	<code>npx playwright test --grep @regression</code>
@confirmation	Re-test d'une anomalie corrigée	<code>npx playwright test --grep @confirmation</code>

Pour un pipeline de déploiement, la suite @smoke doit rester courte et stable. Les suites plus longues peuvent être exécutées la nuit ou avant une release.

4. Intégration Playwright dans Bamboo

Le build publie le projet Playwright comme artifact. Le déploiement QA récupère cet artifact, installe les dépendances nécessaires puis exécute le test.

Étapes essentielles

- Publier le dossier Playwright comme artifact du build.
- Télécharger l'artifact dans le déploiement QA.
- Positionner les variables nécessaires pour viser l'environnement QA.
- Installer les dépendances avec npm ci.
- Installer Chromium si l'agent ne le possède pas déjà.
- Exécuter le test Playwright.
- Faire échouer le déploiement si le test échoue.

Exemple de commande

```
npx playwright test LoginLogout.spec.ts --no-deps --project=chromium
```

Dans le cas étudié, --no-deps permet d'éviter les projets de setup/cleanup qui nécessitent une API REST non encore disponible sur l'environnement. --project=chromium limite l'exécution au navigateur nécessaire dans le pipeline.

5. Exemple de smoke test : connexion / déconnexion

Un test Login/Logout est un bon premier contrôle de déploiement : il vérifie que l'application répond, que l'authentification fonctionne et que l'utilisateur peut terminer sa session.

```
test('Login puis logout @smoke', async ({ page }) => {
  await page.goto(process.env.APP_URL!);

  // Authentification
  await page.getByRole('textbox', { name: "Nom d'utilisateur" })
    .fill(process.env.TEST_USERNAME!);
  await page.getByRole('textbox', { name: 'Mot de passe' })
    .fill(process.env.TEST_PASSWORD!);
  await page.getByRole('button', { name: 'Connexion' }).click();

  // Assertions post-login et logout...
});
```

Les URL internes, comptes, mots de passe et autres informations d'infrastructure doivent rester externalisés dans des variables d'environnement ou dans le gestionnaire de secrets du CI/CD.

6. Validation locale avant le pipeline

Un test destiné à Bamboo doit d'abord être validé localement avec une commande aussi proche que possible de celle du pipeline.

```
npm ci
npx playwright install chromium
npx playwright test LoginLogout.spec.ts --no-deps --project=chromium
```

Le résultat attendu pour le smoke test est simple : le scénario passe avant que la modification soit poussée dans Git et intégrée au pipeline.

7. Bonnes pratiques

- Garder les smoke tests courts, indépendants et fiables.
- Utiliser des sélecteurs accessibles : rôle, libellé ou data-testid plutôt que des classes CSS fragiles.
- Ne pas commiter node_modules, les rapports Playwright ou les résultats d'exécution.
- Ne pas exposer de mots de passe dans le dépôt ou les logs.
- Utiliser npm ci dans le pipeline afin de reconstruire les dépendances depuis package-lock.json.
- Corriger ou retirer du pipeline un test flaky qui pourrait bloquer inutilement les déploiements.
- Créer et nettoyer les données propres à chaque scénario lorsque cela est possible.

8. Évolution progressive du pipeline

L'automatisation peut être enrichie progressivement. L'objectif n'est pas d'exécuter tous les tests après chaque déploiement, mais de placer le bon niveau de contrôle au bon moment.

Étape	Axe	Évolution
1	Smoke	Passer d'un seul LoginLogout à l'ensemble des tests @smoke.
2	Résultats	Publier les résultats avec un reporter JUnit dans Bamboo.
3	Sécurité	Déplacer les secrets vers des variables Bamboo sécurisées.
4	Intégration	Activer les services nécessaires et réintroduire les setups API.
5	Régression	Créer une exécution planifiée de nuit pour @regression.
6	Traçabilité	Relier les résultats à l'outil de gestion des tests, par exemple Zephyr Scale.

9. Points clés à retenir

- Playwright peut servir de garde-fou immédiatement après un déploiement QA.

- Le smoke test doit être rapide : il valide les fonctions essentielles, pas toute la régression.
- La séparation smoke / intégration / système / acceptation facilite la maintenance.
- Le pipeline doit reproduire une exécution déjà validée localement.
- Les secrets et paramètres d'environnement doivent être externalisés.
- La couverture s'élargit progressivement : smoke → résultats → API → régression → traçabilité.

Conclusion

Cette intégration illustre une démarche QA orientée CI/CD : commencer par un contrôle simple et utile, le rendre fiable, puis enrichir progressivement la couverture. Le résultat est un pipeline capable de détecter rapidement une régression critique tout en conservant des temps de déploiement raisonnables.